

vtu-easy.com

DIGITAL SYSTEM DESIGN USING VERILOG (BEC302)

MODULE – 5 VERILOG BEHAVIORAL & STRUCTURAL DESCRIPTION

Topics Covered:

- Behavioral Description Highlights • Structure of Behavioral Description
- Sequential Statements (IF, CASE) • Variable Assignment • Loop Statements
- Behavioral Description of MUX (2:1, 4:1, 8:1) • Highlights of Structural Description
 - Organization of Structural Description • Ripple Carry Adder

Department of ECE
VTU B.E. – 3rd Semester

vtu-easy.com

TABLE OF CONTENTS

Click any topic below to jump directly to that section. vtu-easy.com

- [1. Behavioral Description – Highlights & Overview](#)
 - [1.1 What is Behavioral Description?](#)
 - [1.2 Primary Mechanisms: initial Statement](#)
 - [1.3 Primary Mechanisms: always Statement](#)
- [2. Structure of Verilog Behavioral Description](#)
 - [2.1 Module Structure with always block](#)
 - [2.2 Key Rules for Behavioral Modules](#)
- [3. Sequential Statements](#)
 - [3.1 IF Statement – Syntax & Examples](#)
 - [3.2 IF-ELSE and ELSE-IF Ladder](#)
 - [3.3 CASE Statement – Syntax & Examples](#)
 - [3.4 Signal and Variable Assignment](#)
- [4. Loop Statements](#)
 - [4.1 for Loop](#)
 - [4.2 while Loop](#)
 - [4.3 Verilog repeat Loop](#)
 - [4.4 Verilog forever Loop](#)
- [5. Behavioral Description of Multiplexers](#)
 - [5.1 2x1 MUX – Behavioral \(IF-ELSE & ELSE-IF\)](#)
 - [5.2 4x1 MUX – Behavioral \(CASE Statement\)](#)
 - [5.3 8x1 MUX – Behavioral \(CASE Statement\)](#)
 - [5.4 D-Latch – Behavioral Description](#)
 - [5.5 JK Flip-Flop – Behavioral \(CASE Statement\)](#)
 - [5.6 3-bit Binary Counter – Behavioral](#)
- [6. Highlights of Structural Description](#)
 - [6.1 What is Structural Description?](#)
 - [6.2 Key Facts about Structural Description](#)
- [7. Organization of Structural Description](#)
 - [7.1 Declaration & Instantiation](#)
 - [7.2 Verilog Built-in Gate Primitives](#)
 - [7.3 Verilog Half Adder – Structural](#)
 - [7.4 Full Adder – Structural Description](#)
- [8. Structural Description of Ripple Carry Adder](#)
 - [8.1 1-bit Full Adder Module](#)
 - [8.2 4-bit Ripple Carry Adder using Instantiation](#)
 - [8.3 N-bit Ripple Carry Adder \(Parameterized\)](#)
- [9. Comparison: Behavioral vs Structural vs Dataflow](#)
- [10. Quick Revision – All Topics at a Glance](#)

1. Behavioral Description – Highlights & Overview

1.1 What is Behavioral Description?

Behavioral description is the most powerful and abstract style of hardware description in Verilog. It describes a system in terms of **HOW THE OUTPUTS BEHAVE IN RESPONSE TO INPUT CHANGES** — without specifying the underlying gate-level or transistor-level implementation.

This style is especially useful when:

- The digital logic structure of the system is not yet known or is too complex to generate manually.
- You are designing complex arithmetic units such as multipliers, dividers, and ALUs.
- You are designing computer control units, state machines (FSMs), and processor elements.
- Some biological or analog mechanism is being modeled for simulation purposes.

Key Differences from Other Styles:

Aspect	Dataflow Style	Behavioral Style
Description method	Boolean logic equations using assign	Algorithm / procedural statements using always/initial
Requires logic diagram?	Yes, implicitly	No — only input-output behaviour needed
Statement execution	Concurrent (all at once)	Sequential inside always/initial blocks
Abstraction level	Medium	High (closest to software programming)
Best use case	Combinational circuits	Sequential circuits, FSMs, complex systems

Important Rule: In behavioral description, any signal declared as an output that is assigned a value inside an always or initial block **MUST** be declared as a register (reg). Failure to do so results in a syntax error.

1.2 Primary Mechanism – initial Statement

The **initial statement** is one of the two primary procedural mechanisms used in Verilog behavioral modeling.

Definition: An initial statement executes **EXACTLY ONCE**. It begins execution at time 0 (the very start of simulation) and completes after executing all statements in its body.

Syntax:

```
initial
[timing_control]
procedural_statement;

// OR with multiple statements:
initial begin
[timing_control]
statement1;
statement2;
end
```

Timing Control Options:

- Delay control: #<time_value> — suspends execution for specified time units
- Event control: @(event) — waits until a specified event occurs

- No timing control: statement executes at simulation time 0

Procedural Statements inside initial (or always):

1. Continuous assignment
2. Conditional statement (if-else)
3. Case statement (case-endcase)
4. Loop statement (for, while, repeat, forever)
5. Wait statement
6. Sequential block (begin-end)

Example 1 – initial without timing control:

```
reg yurt;
// ...
initial
    yurt = 2; // yurt is assigned value 2 at time 0, only once
```

In this example, the initial statement executes at time 0 and assigns yurt=2 at time 0. Since there is no timing control, the assignment happens immediately at simulation start and never repeats.

Example 2 – initial with delay (#5):

```
reg curt;
// ...
initial
    #5 curt = 1; // curt is assigned 1 at time=5, execution starts at time=0
```

Here, the initial statement starts execution at time 0, waits for 5 time units due to the #5 delay, and then assigns curt=1 at time 5. The initial statement completes execution at time 5.

Example 3 – initial with multiple statements:

```
reg clk, reset;
initial begin
    clk  = 1'b0; // at time 0
    reset = 1'b1; // at time 0
    #20 reset = 1'b0; // at time 20
end
```

 **NOTE:** initial statements are primarily used in TEST BENCHES to set up initial values, apply stimulus, and control simulation. They are NOT synthesisable.

1.3 Primary Mechanism – always Statement

The **always statement** is the second and more commonly used procedural mechanism in Verilog. It is the heart of behavioral description for synthesisable circuits.

Definition: An always statement executes REPEATEDLY — it loops from start to end, and then begins again. Like initial, it begins execution at time 0.

Syntax:

```
always
    [timing_control]
    procedural_statement;

// OR with multiple statements:
always @(sensitivity_list) begin
    statement1;
    statement2;
end
```

Critical Rule: An always statement MUST have a timing control (delay or event). Without timing control, the statement executes in a zero-time infinite loop, hanging the simulation!

Example 1 – always WITHOUT timing control (WRONG – infinite loop!):

```
always
  clk = ~clk; // WRONG: infinite zero-time loop - simulation hangs!
```

Example 2 – always WITH delay (clock generator):

```
always
  #5 clk = ~clk; // clock with period = 10 time units
```

This always statement toggles clk every 5 time units, producing a clock waveform with a period of 10 time units (5 high + 5 low). This is a commonly used clock generation pattern in testbenches.

Example 3 – always WITH event sensitivity list (combinational):

```
always @(a, b, cin) begin // executes whenever a, b, or cin changes
  sum = a ^ b ^ cin;
  cout = (a&b) | (b&cin) | (a&cin);
end
```

Example 4 – always WITH clock edge (sequential – D Flip-Flop):

```
always @(posedge clk) // executes only on rising edge of clk
  q <= d; // non-blocking assignment
```

Feature	initial Statement	always Statement
Execution count	Executes ONCE at time 0	Executes REPEATEDLY (loops forever)
Timing control	Optional	MANDATORY (else infinite loop)
Primary use	Testbench initialization, stimulus generation	Synthesisable logic (combinational + sequential)
Synthesisable?	NO	YES (with proper sensitivity list)
Begins at	Time 0	Time 0

2. Structure of Verilog Behavioral Description

2.1 Module Structure with always Block

The following is a complete Verilog behavioral description of a half adder using the always block. It demonstrates the standard structure of a behavioral module.

Example – Behavioral Half Adder:

```
module halfadd (a, b, sum, carry);
  input  a, b;           // input ports
  output sum, carry;     // output ports
  reg    sum, carry;     // MUST be declared as reg (assigned inside always)

  always @(a, b)         // sensitivity list: execute when a or b changes
  begin
    #10 sum  = a ^ b;    // XOR for sum, with 10-unit delay
    #10 carry = a & b;    // AND for carry, with 10-unit delay
  end

endmodule
```

Explanation:

- Module name: halfadd. Inputs: a, b. Outputs: sum, carry.
- sum and carry are declared as reg because they are assigned values inside the always block.
- The sensitivity list @(a, b) means the always block executes whenever a or b changes.
- #10 introduces a 10 time-unit propagation delay before each assignment.
- Statements inside begin...end execute sequentially — sum is computed first, then carry.

2.2 Key Rules for Behavioral Modules

Rule 1 – Output signals assigned inside always/initial must be reg:

Any signal that appears on the LEFT-HAND SIDE of an assignment inside an always or initial block must be declared as a register (reg). If it is also declared as output, both declarations are needed.

```
output sum, carry;    // port declaration
reg    sum, carry;    // MUST also be reg since assigned inside always
```

Rule 2 – Sensitivity list must include all RHS signals for combinational logic:

For combinational circuits, the sensitivity list must include ALL signals that appear on the right-hand side. Use always @(*) or always @(a,b,c) to ensure correct simulation.

```
always @(*)           // @(*) = wildcard - includes all RHS signals automatically
  Y = A & B | C;
```

Rule 3 – Use non-blocking assignments (<=) for sequential logic:

For sequential circuits (flip-flops, registers), always use non-blocking assignments (<=) inside clocked always blocks. Blocking assignments (=) are for combinational logic inside always @(*) blocks.

Assignment Type	Operator	Execution	Use For
Blocking	=	Executes immediately (in order, one by one)	Combinational logic inside always @(*)

Assignment Type	Operator	Execution	Use For
Non-blocking	<=	All RHS evaluated first, then all LHS updated simultaneously	Sequential logic inside always @(posedge clk)

 **TIP:** Remember: = for combo (blocking), <= for sequential (non-blocking). Never mix them in the same always block!

3. Sequential Statements

Sequential statements are statements that appear INSIDE always or initial blocks in Verilog. They execute sequentially (one after another) within the block. The major sequential statements are: IF, CASE, and Loop statements.

3.1 IF Statement – Syntax & Examples

The IF statement is the most fundamental sequential statement. It selects one of two (or more) paths for execution based on a Boolean condition. It is the Verilog equivalent of the if-else construct in C/C++.

Definition: An IF statement evaluates a Boolean expression. If the expression is TRUE (non-zero), it executes the "then" statements. If FALSE (zero), it executes the optional "else" statements.

Basic IF Syntax:

```
if (boolean_expression)
begin
    statement1;
    statement2;
    // ... more statements
end
else
begin
    statement_a;
    statement_b;
end
```

Important: The begin...end block is required only when there are multiple statements. For a single statement, begin...end can be omitted — but it is good practice to always include it.

Simple IF Example:

```
if (temp == 1)    // Verilog
    temp = s1;
else
    temp = s2;
```

If temp equals 1, temp is reassigned to s1. Otherwise, temp is assigned s2.

3.2 IF-ELSE and ELSE-IF Ladder

For multiple conditions, an ELSE-IF ladder is used. Conditions are checked sequentially from top to bottom. The FIRST true condition executes its block, and the rest are skipped. This gives PRIORITY to the first condition — important for priority encoders!

ELSE-IF Syntax:

```
if (Boolean_expression1) begin
    statement1;
    statement2;
end
else if (Boolean_expression2) begin
    statement_i;
    statement_ii;
end
else begin
    statement_a;
    statement_b;
end
```


Example – Priority using ELSE-IF:

```

if (signal1 == 1)           // Verilog
    temp = s1;              // signal1 has highest priority
else if (signal2 == 1)
    temp = s2;
else
    temp = s3;              // default (lowest priority)

```

 **NOTE:** ELSE-IF has PRIORITY: condition1 is checked first, then condition2, etc. The first TRUE condition wins. This is fundamentally different from CASE statement which checks all conditions simultaneously (equal priority).

2x1 MUX using IF-ELSE (with tristate output):

```

module mux2x1 (A, B, SEL, Gbar, Y);
    input  A, B, SEL, Gbar;
    output Y;
    reg    Y;

    always @(SEL, A, B, Gbar)
    begin
        if (Gbar == 1)           // if enable is HIGH (disabled)
            Y = 1'bz;           // output is high impedance (tristate)
        else                     // enable is LOW (active)
            begin
                if (SEL)
                    Y = B;       // SEL=1: select input B
                else
                    Y = A;       // SEL=0: select input A
            end
        end
    end
endmodule

```

2x1 MUX using ELSE-IF:

```

module MUXBH (A, B, SEL, Gbar, Y);
    input  A, B, SEL, Gbar;
    output Y;
    reg    Y;

    always @(SEL, A, B, Gbar)
    begin
        if      (Gbar==0 & SEL==1) Y = B;    // Gbar LOW, SEL=1: Y=B
        else if (Gbar==0 & SEL==0) Y = A;    // Gbar LOW, SEL=0: Y=A
        else      Y = 1'bz;                // Gbar HIGH: high-Z
    end
endmodule

```

3.3 CASE Statement – Syntax & Examples

The CASE statement is a multi-way branch statement that selects one of several execution paths based on the value of an expression. It is the behavioral equivalent of the selected signal assignment (with-select in VHDL).

Key Difference from IF-ELSE: In a CASE statement, ALL choices are compared simultaneously with the case expression — there is NO priority among choices. This makes it ideal for truth-table-style descriptions like multiplexers, encoders, and decoders.

CASE Syntax:

```

case (expression)
  choice1 : statement1;
  choice2 : statement2;
  choice3 : begin
    statement_a;
    statement_b;
  end
  default : default_statement; // handles all unspecified cases
endcase

```

Rules for CASE statement:

- Each choice must be a constant or constant expression — NOT a variable
- All choices must be mutually exclusive (no value can appear twice)
- A "default" clause should always be included to handle unspecified/unknown values
- Without a default, unmatched cases leave the output unchanged — which may infer unwanted latches in synthesis
- casex and casez are variants that allow don't-cares in the case expression

Example – 4:1 MUX using CASE:

```

module mux4x1 (A, B, C, D, SEL, Y);
  input  A, B, C, D;
  input  [1:0] SEL;
  output Y;
  reg    Y;

  always @(*)
  begin
    case (SEL)
      2'b00 : Y = A; // SEL=0 → select A
      2'b01 : Y = B; // SEL=1 → select B
      2'b10 : Y = C; // SEL=2 → select C
      2'b11 : Y = D; // SEL=3 → select D
      default: Y = 1'bx; // safety default
    endcase
  end
endmodule

```

JK Flip-Flop using CASE (Example 3):

A positive-edge-triggered JK flip-flop has the following truth table: JK=00 (hold), JK=01 (reset), JK=10 (set), JK=11 (toggle).

```

module JK_FF (JK, clk, q, qb);
  input  [1:0] JK;
  input  clk;
  output q, qb;
  reg    q, qb;

  always @(posedge clk) // triggered on rising edge of clock
  begin
    case (JK)
      2'd0 : q = q; // JK=00: Hold (no change)
      2'd1 : q = 1'b0; // JK=01: Reset (q=0)
      2'd2 : q = 1'b1; // JK=10: Set (q=1)
      2'd3 : q = ~q; // JK=11: Toggle (q inverts)
    endcase
    qb = ~q; // qb is complement of q
  end
endmodule

```

```
endmodule
```

JK Input	Operation	Output Q
00	Hold – no change	Q (unchanged)
01	Reset	0
10	Set	1
11	Toggle	~Q (inverted)

3.4 Signal and Variable Assignment

In Verilog behavioral description, there are two types of assignment statements used inside always/initial blocks:

1. Blocking Assignment (=):

A blocking assignment executes immediately and in sequence — the simulator WAITS for the current assignment to complete before moving to the next statement. The LHS is updated before the next statement begins.

```
// Blocking assignment example:
always @(*) begin
    temp = a & b;    // executes first, temp gets new value
    Y     = temp | c; // executes second, uses NEW value of temp
end
```

2. Non-Blocking Assignment (<=):

A non-blocking assignment does NOT block execution. ALL right-hand sides are evaluated first using the current values, and THEN all left-hand sides are updated simultaneously at the end of the time step.

```
// Non-blocking assignment example (swap using <=):
always @(posedge clk) begin
    a <= b;    // a gets OLD value of b
    b <= a;    // b gets OLD value of a
end           // both update simultaneously: correct swap!
```

Why this matters – D-Latch Example:

```
module D_latch (d, E, Q, Qb);
    input  d, E;
    output Q, Qb;
    reg    Q, Qb;

    always @(d, E)
    begin
        if (E == 1) begin    // latch is transparent when E=1
            Q = d;           // Q follows d
            Qb = ~Q;         // Qb = NOT Q (uses UPDATED Q due to blocking =)
        end
        // when E=0, Q and Qb hold their last value (latch behavior)
    end
endmodule
```

Since blocking assignment is used, when Q=d executes, the next line Qb=~Q uses the NEW (just-updated) value of Q. This gives the correct latch behavior.

Feature	Blocking (=)	Non-Blocking (<=)
Execution	Immediate and sequential — waits before next statement	Deferred — all RHS evaluated first, then LHS updated
Effect on next line	Next statement sees UPDATED value	Next statement sees OLD value
Used for	Combinational logic inside always @(*)	Sequential/clocked logic inside always @(posedge clk)
Swap operation	Cannot swap without temp variable	Can swap directly: a<=b; b<=a;
Risk	Can cause race conditions in clocked blocks	Safe for clocked blocks

4. Loop Statements

Loop statements in Verilog are used to REPEAT the execution of a set of statements multiple times. They are sequential statements and must appear inside always or initial blocks. Loop statements help reduce code size and improve readability.

Verilog loop types:

- for – counted loop with initialization, condition, and step (similar to C)
- while – condition-controlled loop (loop while condition is true)
- repeat – fixed-count loop (Verilog-specific)
- forever – infinite loop (Verilog-specific, used mainly in testbenches)

4.1 for Loop

The for loop is the most commonly used loop in Verilog. It has three parts: initialization, condition check, and step assignment — exactly like the C language for loop.

Syntax:

```
for (initial_assignment; condition; step_assignment)
begin
    statements;
end
```

Execution Flow:

7. initial_assignment executes once at the start
8. condition is checked — if TRUE, execute loop body; if FALSE, exit loop
9. After loop body executes, step_assignment is applied
10. Go back to step 2 and repeat

Example – for loop in Verilog:

```
for (i=0; i<=2; i=i+1)
begin
    if (temp[i] == 1)
    begin
        result = result + 2;
    end
end
```

This loop iterates with $i = 0, 1, 2$. Each iteration checks if bit i of temp is 1, and if so, adds 2 to result. This is equivalent to a popcount-weighted sum.

Example – for loop for register initialization:

```
integer i;
reg [7:0] mem [0:255];    // 256-byte memory

initial begin
    for (i=0; i<256; i=i+1)
        mem[i] = 8'b0;    // initialise all memory to 0
end
```

 **NOTE:** In Verilog, the loop variable (i) should be declared as an integer. Loop variables are not allowed to be wire or reg types.

4.2 while Loop

The while loop is a condition-controlled loop. The loop body executes repeatedly AS LONG AS the specified condition evaluates to TRUE. When the condition becomes FALSE, execution exits the loop.

Syntax:

```
while (condition)
begin
    statement1;
    statement2;
end
```

Execution Flow:

11. Evaluate condition — if TRUE, execute loop body; if FALSE, exit loop
12. Execute all statements in the loop body
13. Go back to step 1

Critical Rule: The statements inside the while loop MUST eventually cause the condition to become FALSE, or the loop will run infinitely (infinite loop), hanging the simulation.

Example – while loop:

```
integer count;

initial begin
    count = 0;
    while (count < 10) begin
        #10 count = count + 1;    // increment count every 10 time units
    end
    // exits when count reaches 10
end
```

Example – while loop for bit counting:

```
integer temp_data;
integer ones_count;

initial begin
    ones_count = 0;
    temp_data = data_in;
    while (temp_data > 0) begin
        if (temp_data[0] == 1)
            ones_count = ones_count + 1;
        temp_data = temp_data >> 1;    // right shift
    end
    // ones_count now holds the number of 1-bits in data_in
end
```

4.3 Verilog repeat Loop

The repeat loop is a Verilog-specific loop construct. It executes the loop body a FIXED number of times. Unlike for and while, the repeat construct does NOT use a condition or index variable — it simply repeats a fixed number of times.

Key Properties:

- The repeat count must be a CONSTANT, variable, or signal — NOT an expression or condition.
- The count is evaluated ONCE at the beginning of the loop and does not change during execution.
- Cannot be used with conditional expressions.

Syntax:

```
repeat (count)
```

```
begin
    statement1;
    statement2;
end
```

Example:

```
repeat (32)
begin
    #100 i = i + 1;    // executes exactly 32 times
end
```

In this example, the statement $i=i+1$ is executed exactly 32 times. Each execution waits 100 time units before incrementing i . After 32 iterations (at time $32 \times 100 = 3200$), the loop exits.

Practical Use – Clock generation with repeat:

```
initial begin
    clk = 0;
    repeat (20) #10 clk = ~clk;    // generate 10 clock cycles (20 toggles)
end
```

 **TIP:** repeat is cleaner than for when you just need to repeat N times without needing a loop variable index inside the body.

4.4 Verilog forever Loop

The forever loop is another Verilog-specific construct. As the name implies, it executes its body ENDLESSLY — it is an infinite loop with no termination condition.

Key Properties:

- Does NOT contain any condition or count expression
- Runs until the simulation is terminated by the \$finish system task
- Can be exited early using the "disable" statement
- Equivalent to "while (1)" — while loop with condition that always evaluates to TRUE
- Most common use: generating clock waveforms in testbenches

Syntax:

```
forever
begin
    statements;
end
```

Example – Clock generation using forever:

```
initial begin
    clk = 1'b0;                // initialise clock to 0
    forever #20 clk = ~clk;    // toggle every 20 units → period = 40 units
end
```

This is the standard way to generate a clock in a Verilog testbench. The initial block sets $clk=0$, and then forever ensures the clock toggles every 20 time units indefinitely.

Loop Type	Termination	Count Variable	Verilog Specific?	Primary Use
for	When condition becomes false	Yes (index variable)	No (common to C,VHDL)	Counted iterations
while	When condition becomes false	No (implicit in condition)	No	Condition-based looping

Loop Type	Termination	Count Variable	Verilog Specific?	Primary Use
repeat	After fixed N iterations	No (just a count)	YES	Fixed-count repetition
forever	Never (until \$finish or disable)	No	YES	Infinite loops, clock gen

5. Behavioral Description of Multiplexers

Multiplexers are one of the most fundamental combinational circuits. Behavioral description is a clean and compact way to describe MUX circuits of any size. Here we cover 2:1, 4:1, and 8:1 MUX along with the D-Latch, JK Flip-Flop, and Binary Counter.

5.1 2:1 MUX – Behavioral Description

A 2:1 multiplexer selects one of two data inputs (A or B) based on the select line (SEL). Gbar is an active-low enable input — when Gbar=1, the output is high-impedance (tristate).

Truth Table for 2:1 MUX with Active-Low Enable:

Gbar (Enable_bar)	SEL	Output Y
1 (disabled)	X (don't care)	Z (High Impedance)
0 (enabled)	0	A (select input A)
0 (enabled)	1	B (select input B)

Verilog Code – 2:1 MUX using IF-ELSE:

```
module mux2x1_if (A, B, SEL, Gbar, Y);
    input  A, B, SEL, Gbar;
    output Y;
    reg    Y;

    always @(SEL, A, B, Gbar)
    begin
        if (Gbar == 1)           // active-low enable: disabled when Gbar=1
            Y = 1'bz;           // high impedance output
        else begin
            if (SEL)             // SEL=1: select B
                Y = B;
            else                 // SEL=0: select A
                Y = A;
        end
    end
endmodule
```

Verilog Code – 2:1 MUX using ELSE-IF:

```
module mux2x1_elseif (A, B, SEL, Gbar, Y);
    input  A, B, SEL, Gbar;
    output Y;
    reg    Y;

    always @(SEL, A, B, Gbar)
    begin
        if      (Gbar==0 && SEL==1) Y = B;
        else if (Gbar==0 && SEL==0) Y = A;
        else                                Y = 1'bz;
    end
endmodule
```

5.2 4:1 MUX – Behavioral (CASE Statement)

A 4:1 multiplexer selects one of four inputs using a 2-bit select line. The CASE statement is the most natural and readable way to implement this.

Truth Table:

SEL[1:0]	Output Y
00	A
01	B
10	C
11	D

Verilog Code – 4:1 MUX using CASE:

```

module mux4x1_case (A, B, C, D, SEL, Y);
    input  A, B, C, D;
    input  [1:0] SEL;
    output Y;
    reg     Y;

    always @(*)
    begin
        case (SEL)
            2'b00 : Y = A;
            2'b01 : Y = B;
            2'b10 : Y = C;
            2'b11 : Y = D;
            default: Y = 1'bx; // safety: unknown select
        endcase
    end
endmodule

```

5.3 8:1 MUX – Behavioral (CASE Statement)

An 8:1 multiplexer selects one of eight inputs using a 3-bit select line. Using CASE, all 8 combinations are described cleanly.

```

module mux8x1_case (I, SEL, Y);
    input  [7:0] I; // 8 data inputs
    input  [2:0] SEL; // 3-bit select
    output Y;
    reg     Y;

    always @(*)
    begin
        case (SEL)
            3'd0 : Y = I[0];
            3'd1 : Y = I[1];
            3'd2 : Y = I[2];
            3'd3 : Y = I[3];
            3'd4 : Y = I[4];
            3'd5 : Y = I[5];
            3'd6 : Y = I[6];
            3'd7 : Y = I[7];
            default: Y = 1'bx;
        endcase
    end
endmodule

```

5.4 D-Latch – Behavioral Description

A D-Latch is a level-sensitive storage element. When Enable (E) is HIGH (1), the output Q follows input D. When E goes LOW (0), Q holds (latches) its last value.

```
module D_latch (d, E, Q, Qb);
    input  d, E;
    output Q, Qb;
    reg    Q, Qb;

    always @(d, E)          // trigger on any change of d or E
    begin
        if (E == 1) begin    // latch is transparent when E=1
            Q = d;           // output follows input
            Qb = ~Q;         // complement output
        end
        // when E=0: Q and Qb hold their values (no assignment → latch)
    end
endmodule
```

Key Point: When the if(E==1) condition is FALSE (E=0), no assignment is made to Q and Qb. Verilog infers that they should HOLD their previous values — synthesizing a LATCH.

5.5 JK Flip-Flop – Behavioral (already covered in Section 3.3)

Refer to Section 3.3 for the complete JK Flip-Flop behavioral description using the CASE statement. Key points:

- Triggered on POSEDGE (rising edge) of clock — edge-sensitive, not level-sensitive
- Four states: Hold (JK=00), Reset (JK=01), Set (JK=10), Toggle (JK=11)
- Output qb = ~q is updated inside the always block after the case statement

5.6 3-bit Binary Counter – Behavioral (CASE Statement)

A 3-bit synchronous binary counter increments on each rising clock edge. With synchronous clear (clr): when clr=0, counter increments; when clr=1, counter resets to 000.

```
module CT_CASE (clk, clr, q);
    input      clk, clr;
    output [2:0] q;
    reg [2:0] q;

    initial
        q = 3'b101;          // initial state = 5 (for testing)

    always @(posedge clk)
    begin
        if (clr == 0) begin    // clr=0: count up
            case (q)
                3'd0 : q = 3'd1;
                3'd1 : q = 3'd2;
                3'd2 : q = 3'd3;
                3'd3 : q = 3'd4;
                3'd4 : q = 3'd5;
                3'd5 : q = 3'd6;
                3'd6 : q = 3'd7;
                3'd7 : q = 3'd0; // wrap around
            endcase
        end
        else
            q = 3'b000;         // clr=1: synchronous clear
        end
    end
endmodule
```

The initial statement sets $q=101$ (5) for testing. The counter uses a CASE to explicitly list each state transition. The synchronous clear overrides counting when $clr=1$.

6. Highlights of Structural Description

6.1 What is Structural Description?

Structural description is a method of defining a system by specifying its BASIC HARDWARE COMPONENTS and their INTERCONNECTIONS. Instead of describing what the system does (behavioral), it describes what the system IS MADE OF — like a schematic diagram expressed in code.

Analogy: Structural description is to HDL what a circuit schematic is to a hardware designer. Just as a schematic shows individual gates connected by wires, a structural description instantiates gates and modules connected by signal nets.

Best Use Cases:

- When the hardware components (gates, sub-modules) are already known and well-defined
- When you need FULL CONTROL over the specific implementation (e.g., choosing ripple carry vs. carry-lookahead adder)
- When designing at gate level for timing optimization or layout-aware design
- When building hierarchical designs by connecting pre-designed blocks
- When performing gate-level simulation after synthesis

6.2 Key Facts about Structural Description

Fact 1 – Components and Abstraction Levels:

Structural description can describe a system at different levels of abstraction:

- Gate level: uses basic logic gates (AND, OR, NOT, XOR, NAND, NOR, XNOR)
- Register Transfer Level (RTL): uses registers, MUXes, ALUs as building blocks
- Processor level: uses entire functional units like CPUs, memory controllers

Fact 2 – Design Specificity vs Behavioral:

Structural description gives the designer CONTROL over the implementation that behavioral description does not offer.

- Behavioral: writing $C = A + B$ lets the synthesis tool choose the adder type
- Structural: you explicitly instantiate a carry-lookahead adder, a ripple carry adder, or any specific architecture

Fact 3 – All Statements are Concurrent:

ALL statements in a structural description are CONCURRENT — they all execute simultaneously whenever any of their input signals change. This is similar to dataflow description. This naturally models the fact that all physical gates in a circuit operate in parallel.

Fact 4 – Verilog vs VHDL Gate Support:

A major difference between Verilog and VHDL in structural description:

- Verilog: has built-in primitive gates (and, or, xor, not, nand, nor, xnor, buf, etc.) — ready to use directly without any imports.
- VHDL: basic packages do NOT include any gate primitives. You must link the architecture to external libraries, packages, or modules that contain gate descriptions.

Aspect	Behavioral	Structural
What is described	Algorithm / function	Components and connections
Knowledge required	Input-output behavior only	Must know the circuit structure
Statement type	Sequential (inside always/initial)	Concurrent (all run simultaneously)
Implementation control	Low (synthesis tool decides)	High (designer specifies components)
Code style	Similar to software programming	Similar to schematic drawing
Typical use	Complex systems, FSMs, counters	Gate-level, RTL, hierarchical design

7. Organization of Structural Description

7.1 Declaration & Instantiation

A structural description has two key parts in its architecture/module body:

Part 1 – Declaration:

In VHDL, the declaration part declares all components used in the design. Each component must be declared with its input/output ports before it can be used (instantiated). In Verilog, built-in gate primitives do NOT need declaration — they are automatically available.

VHDL Component Declaration Example:

```
component xor2
  port (i1, i2 : in  std_logic;
        o1      : out std_logic);
end component;
```

The xor2 component has two inputs (i1, i2) and one output (o1). Once declared, it can be instantiated (used) multiple times.

Part 2 – Instantiation:

Instantiation is the process of placing a component in a design and connecting its ports to actual signals. Each instantiation creates a unique copy of the component (a logic element in the circuit).

VHDL Instantiation Example:

```
X1: xor2 port map (A, B, sum);
```

This maps: input A → port i1 of xor2, input B → port i2 of xor2, output sum → port o1 of xor2. The logic relationship between A, B, and sum is the same as between i1, i2, and o1 of the xor2 component.

7.2 Verilog Built-in Gate Primitives

Verilog provides a comprehensive set of built-in gate primitives. These gates are ready to use in any module without any declaration or import. They are instantiated using a simple statement:

General Gate Instantiation Syntax:

```
<gate_type> [instance_name] (output, input1, input2, ...);
```

Important: In Verilog, the OUTPUT is ALWAYS listed FIRST, followed by the inputs. The instance name (e.g., X1, G1) is OPTIONAL — it can be omitted.

Example:

```
xor X1 (sum, a, b);    // XOR gate: output=sum, inputs=a,b; X1 is optional
                        identifier
and  G1 (cout, a, b);  // AND gate: output=cout, inputs=a,b
xor   (sum, a, b);     // same XOR gate without instance name
```

Gate Keyword	Operation	Example
and	N-input AND gate	and (Y, A, B, C); // 3-input AND
or	N-input OR gate	or (Y, A, B);

Gate Keyword	Operation	Example
xor	N-input XOR gate	xor (Y, A, B);
not	1-input NOT gate (inverter)	not (Y, A); // Y = ~A
nand	N-input NAND gate	nand (Y, A, B);
nor	N-input NOR gate	nor (Y, A, B);
xnor	N-input XNOR gate	xnor (Y, A, B);
buf	Buffer (non-inverting)	buf (Y, A); // Y = A
bufif1	Tristate buffer, enable-high	bufif1 (Y, A, EN);
bufif0	Tristate buffer, enable-low	bufif0 (Y, A, EN);

7.3 Verilog Half Adder – Structural

A structural description of a half adder directly instantiates an XOR gate (for sum) and an AND gate (for carry).

```
module half_add (a, b, S, C);
    input  a, b;
    output S, C;
    xor (S, a, b);    // S = a XOR b (sum)
    and (C, a, b);    // C = a AND b (carry)
endmodule
```

Structural description (Program 4.1 / System module):

```
module system (a, b, sum, cout);
    input  a, b;
    output sum, cout;
    xor X1 (sum, a, b);    // X1 is optional identifier; can be omitted
    and a1 (cout, a, b);   // a1 is optional identifier; can be omitted
endmodule
```

7.4 Full Adder – Structural Description

A full adder computes sum and carry-out from three inputs: a, b, and carry-in (cin). It can be built structurally using XOR, AND, and OR gates.

Logic Equations:

Sum = a XOR b XOR cin

Carry = (a AND b) OR (b AND cin) OR (a AND cin)

Verilog Structural Full Adder:

```
module full_adder (a, b, cin, sum, cout);
    input  a, b, cin;
    output sum, cout;
    wire  s1, c1, c2, c3;    // internal wires

    xor G1 (s1, a, b);        // s1 = a XOR b
    xor G2 (sum, s1, cin);    // sum = s1 XOR cin = a XOR b XOR cin

    and G3 (c1, a, b);        // c1 = a AND b
    and G4 (c2, b, cin);      // c2 = b AND cin
    or  G5 (cout, c1, c2);    // cout = c1 OR c2
```



```
    and G5 (c3, a,  cin);    // c3 = a AND cin

    or  G6 (cout, c1, c2, c3); // cout = c1 OR c2 OR c3
endmodule
```

8. Structural Description of Ripple Carry Adder

A Ripple Carry Adder (RCA) is an N-bit adder built by cascading N full adder stages in series. The carry OUTPUT of one stage is connected to the carry INPUT of the next stage — the carry "ripples" from the least significant bit (LSB) to the most significant bit (MSB), giving the circuit its name.

Architecture:

- Stage 0 (LSB): inputs $a[0]$, $b[0]$, $cin=0 \rightarrow$ outputs $sum[0]$, carry to stage 1
- Stage 1: inputs $a[1]$, $b[1]$, cin =carry from stage 0 $\rightarrow$ outputs $sum[1]$, carry to stage 2
- Stage N-1 (MSB): inputs $a[N-1]$, $b[N-1]$, cin =carry from stage N-2 $\rightarrow$ outputs $sum[N-1]$, $cout$

Key Characteristics of Ripple Carry Adder:

Property	Details
Structure	N full adder stages cascaded in series
Carry propagation	Carry ripples from LSB to MSB — sequential propagation
Delay	$O(N)$ — delay increases linearly with the number of bits
Hardware cost	Low — N full adders, minimal extra logic
Advantage	Simple, easy to design, low hardware cost
Disadvantage	Slow for large N due to ripple delay
Alternative	Carry Lookahead Adder (CLA) — $O(\log N)$ delay but more hardware

8.1 1-bit Full Adder Module (Building Block)

The basic building block for the ripple carry adder is the 1-bit full adder module. This is exactly the structural full adder from Section 7.4.

```
module full_adder_1bit (a, b, cin, sum, cout);
    input  a, b, cin;
    output sum, cout;
    wire   s1, c1, c2, c3;

    xor G1 (s1, a, b);
    xor G2 (sum, s1, cin);
    and G3 (c1, a, b);
    and G4 (c2, b, cin);
    and G5 (c3, a, cin);
    or  G6 (cout, c1, c2, c3);
endmodule
```

8.2 4-bit Ripple Carry Adder using Instantiation

A 4-bit ripple carry adder is built by instantiating four full adder modules and connecting carries in series. The carry-out (cout) of each stage connects to the carry-in (cin) of the next stage.

```
module rca_4bit (a, b, cin, sum, cout);
    input  [3:0] a, b;    // 4-bit inputs
    input      cin;      // carry in to LSB stage
    output [3:0] sum;     // 4-bit sum output
    output      cout;     // carry out from MSB stage
    wire   c1, c2, c3;    // internal carry wires between stages

    // Stage 0 (LSB): bits a[0], b[0]
    full_adder_1bit FA0 (a[0], b[0], cin, sum[0], c1);
```

```
// Stage 1: bits a[1], b[1]
full_adder_1bit FA1 (a[1], b[1], c1, sum[1], c2);

// Stage 2: bits a[2], b[2]
full_adder_1bit FA2 (a[2], b[2], c2, sum[2], c3);

// Stage 3 (MSB): bits a[3], b[3]
full_adder_1bit FA3 (a[3], b[3], c3, sum[3], cout);

endmodule
```

The internal wires c1, c2, c3 carry the intermediate carry signals between stages. FA0 to FA3 are instance names for the four full adder instantiations. The first argument is always the output in Verilog gate/module instantiation.

 **NOTE:** In Verilog module instantiation (unlike gate instantiation), the argument order follows the port list order as defined in the module. Always match ports in the same order they are declared.

8.3 N-bit Ripple Carry Adder (Parameterized)

Using Verilog parameters and generate blocks, we can create a flexible N-bit ripple carry adder that works for any word width.

```
module rca_Nbit #(parameter N=4) (a, b, cin, sum, cout);
    input  [N-1:0] a, b;
    input          cin;
    output [N-1:0] sum;
    output          cout;

    wire [N:0] carry;          // carry[0]=cin, carry[N]=cout
    assign carry[0] = cin;
    assign cout    = carry[N];

    genvar i;
    generate
        for (i=0; i<N; i=i+1) begin : FA_STAGE
            full_adder_1bit FA (a[i], b[i], carry[i], sum[i], carry[i+1]);
        end
    endgenerate
endmodule
```

By changing the parameter N, this module can be configured as a 4-bit, 8-bit, 16-bit, or 32-bit adder without any code modification.

9. Comparison: Behavioral vs Structural vs Dataflow

Feature	Behavioral	Structural	Dataflow
Primary keyword	always / initial	and/or/xor/module instantiation	assign
Statement type	Sequential (inside blocks)	Concurrent (all run together)	Concurrent (all run together)
Abstraction level	High (algorithm)	Low (gate/component level)	Medium (Boolean equations)
Need to know circuit?	No — only behavior	Yes — must know exact structure	Partially — need Boolean equations
Implementation control	Low (tool decides structure)	High (designer specifies gates)	Medium
Best for	FSMs, counters, complex units	Hierarchical design, gate-level	Combinational logic
Output declaration	Must be reg if assigned in block	Usually wire (driven by gates)	Usually wire (driven by assign)
Typical circuits	D FF, JK FF, counters, MUX	RCA, hierarchical systems	Adders, MUX, comparators
Synthesisable?	Yes (always @*) / No (initial)	Yes	Yes

Quick Decision Guide:

- Use BEHAVIORAL when: you know what the circuit does but not how to build it with gates.
- Use STRUCTURAL when: you want full control over the circuit architecture and know the components.
- Use DATAFLOW when: you can express the output as a Boolean expression of inputs.
- Use MIXED when: different parts of a large design benefit from different styles.

10. Quick Revision – All Topics at a Glance

 Complete exam-focused summary of all Module 5 topics.

10.1 Behavioral Description – Key Points

Topic	Key Point
Definition	Describes system behavior (outputs vs inputs) — structure not needed
initial statement	Executes ONCE at time 0 — used for testbench init — NOT synthesisable
always statement	Executes REPEATEDLY — needs timing control — SYNTHESISABLE
Sensitivity list	@(a,b) or @(*) — controls when always block executes
reg declaration	Output signals assigned inside always/initial MUST be declared reg
Blocking (=)	Sequential execution — immediate update — for combinational always @(*)
Non-blocking (<=)	Concurrent-style update — all RHS computed first — for clocked blocks
always without timing	NEVER do always clk=~clk; — infinite loop, hangs simulation!

10.2 Sequential Statements – Key Points

Statement	Key Point
IF	Checks conditions sequentially — first TRUE condition wins — gives PRIORITY
ELSE-IF	Multiple conditions checked in order — first match executes — priority encoder style
CASE	All choices compared simultaneously — NO priority — truth table style
casex/casez	CASE with X/Z don't-cares in patterns — useful for FSMs
default in CASE	Always include default — prevents latch inference in synthesis

10.3 Loop Statements – Key Points

Loop	Key Point
for	for(i=0; i<N; i=i+1) — needs integer variable — most common in synthesis
while	while(condition) — must ensure condition eventually becomes false
repeat	repeat(N) — executes exactly N times — no index variable — Verilog specific
forever	forever — infinite loop — clock generation in testbenches — Verilog specific
All loops	Must be inside always or initial blocks — sequential execution

10.4 Structural Description – Key Points

Topic	Key Point
Definition	Describes system as interconnected components/gates
When to use	When hardware structure is known and specific architecture is needed

Topic	Key Point
Verilog gates	Built-in: and, or, xor, not, nand, nor, xnor, buf — no import needed
Gate syntax	gate_type [name] (output, input1, input2); — OUTPUT ALWAYS FIRST
Instance name	Optional in Verilog gate instantiation (e.g., xor X1 or just xor)
Module instantiation	module_name instance_name (connections); — positional or named port mapping
All concurrent	All gate/module instances run simultaneously — no sequential execution

10.5 Ripple Carry Adder – Key Points

Aspect	Key Point
Structure	N full adders cascaded — carry ripples from LSB to MSB
Delay	$O(N)$ — proportional to number of bits — slow for large N
Verilog	Build FA module first, then instantiate N times with carry connections
Internal carry	Wire array or individual wires for inter-stage carry signals
Parameterized	Use parameter N and generate blocks for flexible N-bit adder
Alternative	Carry Lookahead Adder — $O(\log N)$ delay but more complex

10.6 Must-Know Code Snippets

1. Clock generator (always + forever):

```
initial begin clk=0; forever #10 clk=~clk; end // period=20
```

2. D Flip-Flop (behavioral):

```
always @(posedge clk) q <= d;
```

3. 2:1 MUX (IF-ELSE):

```
always @(*) if(SEL) Y=B; else Y=A;
```

4. 4:1 MUX (CASE):

```
always @(*) case(SEL) 2'b00:Y=A; 2'b01:Y=B; 2'b10:Y=C; 2'b11:Y=D; endcase
```

5. Structural Half Adder:

```
xor(S,a,b); and(C,a,b);
```

6. 4-bit RCA instantiation:

```
full_adder FA0(a[0],b[0],cin,sum[0],cl); // repeat for FA1,FA2,FA3
```

7. JK FF (CASE):

```
always @(posedge clk) case(JK) 0:q=q; 1:q=0; 2:q=1; 3:q=~q; endcase
```

8. for loop:

```
for(i=0; i<N; i=i+1) begin ... end
```

10.7 Top Exam Questions

- Explain the highlights of Verilog behavioral description. Differentiate initial and always statements. (8 marks)
- Explain the structure of Verilog behavioral description with a half adder example. (6 marks)
- Explain IF and CASE statements with syntax and examples. Differentiate IF-ELSE and CASE. (8 marks)

17. Explain the difference between blocking and non-blocking assignment with examples. (6 marks)
 18. Explain for, while, repeat, and forever loop statements in Verilog with examples. (8 marks)
 19. Write Verilog behavioral description of 2:1 MUX using IF-ELSE and ELSE-IF. (6 marks)
 20. Write Verilog behavioral description of 4:1 MUX using CASE statement. (6 marks)
 21. Write Verilog behavioral description of JK flip-flop. (6 marks)
 22. Explain the highlights of Verilog structural description. Compare with behavioral description. (6 marks)
 23. Explain the organization of structural description with a half adder example. (6 marks)
 24. Write Verilog structural description of a 4-bit ripple carry adder using full adder modules. (8 marks)
 25. Explain Verilog built-in gate primitives with syntax and list of gates. (4 marks)
-

END OF MODULE 5 – ALL THE BEST FOR YOUR EXAMS! 🎓

vtu-easy.com